

Machine Learning

8. *Supervised Learning* 4

Sunwoo Lee
sunwool@inha.ac.kr



INHA UNIVERSITY

Today, we will learn...

- Supervised Learning 4
 - One of the simplest ML algorithms, K-Nearest Neighbors (KNN)
 - Regression / Classification with KNN



Do not share or distribute these slides!

저작권 안내 / Copyright Notice

본 사이트에서 수업 자료로 이용되는 저작물은 저작권법 제25조 수업목적저작물 이용 보상금제도의 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. **약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개·공유 및 수업 목적외의 사용을 금지합니다. (일부 공개로 제공되는 유튜브 강의를 불법으로 공유하거나 배포하는 행위를 금함)**

The teaching materials provided on this site are copyrighted and signed legally by the Korea Reproduction and Transmission Rights Association. Using these materials beyond the scope of the agreement is a violation against copyright laws. **Hence, one is not allowed to open or share the materials in public. Using them outside of the class is strictly prohibited.**

2020. 3. 11.

인하대학교·한국복제전송저작권협회

Inha University · Korea Reproduction and Transmission Rights Association

- ***K-Nearest Neighbors Algorithm***
- Eager vs. Lazy Learning Methods

Supervised Learning Algorithms

- **Model**-based Learning

- Linear / logistic regression
- Decision trees
- Perceptron, neural networks
- Support Vector Machine

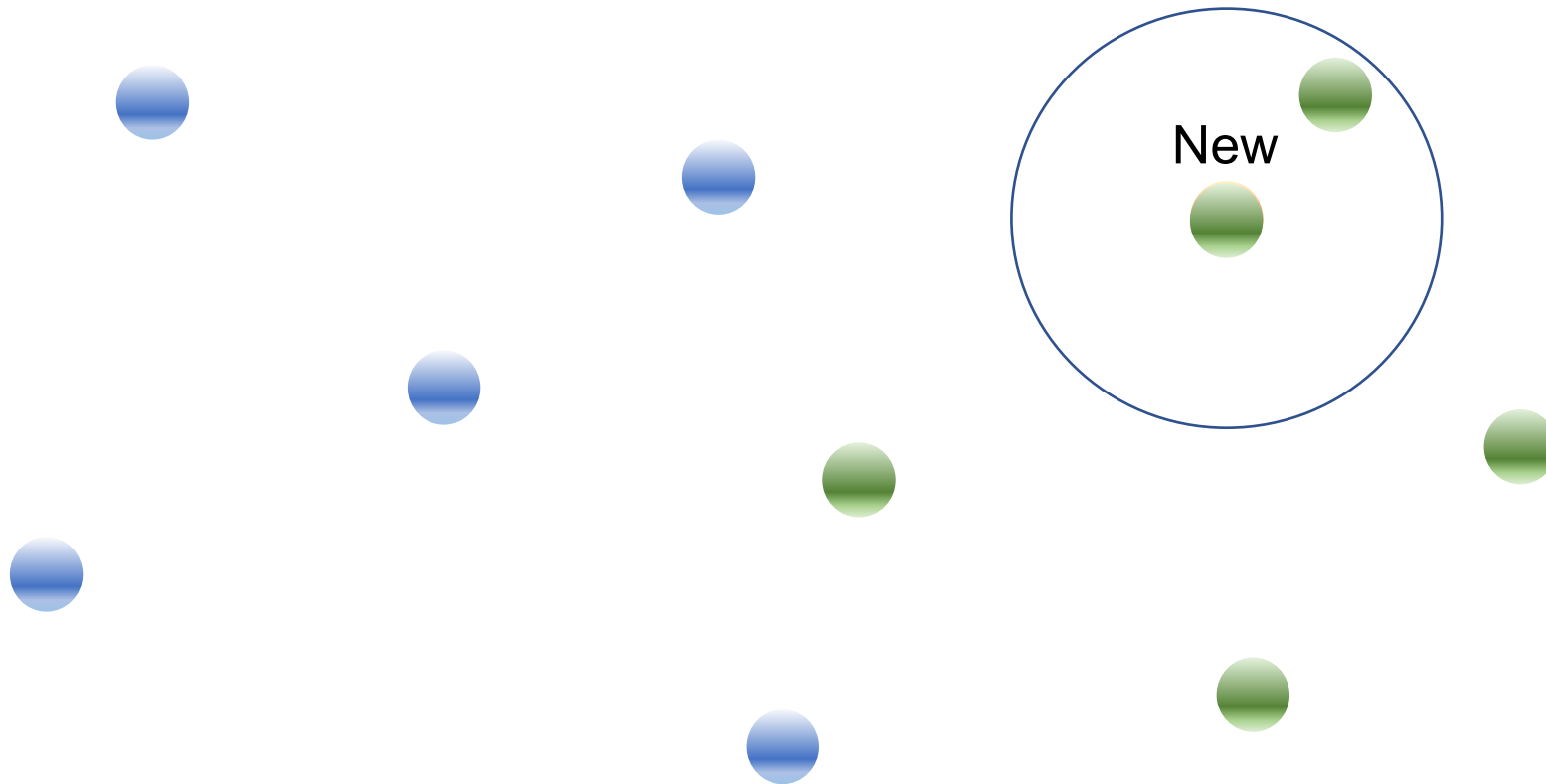
Build up a model from the given data.

- **Instance**-based Learning

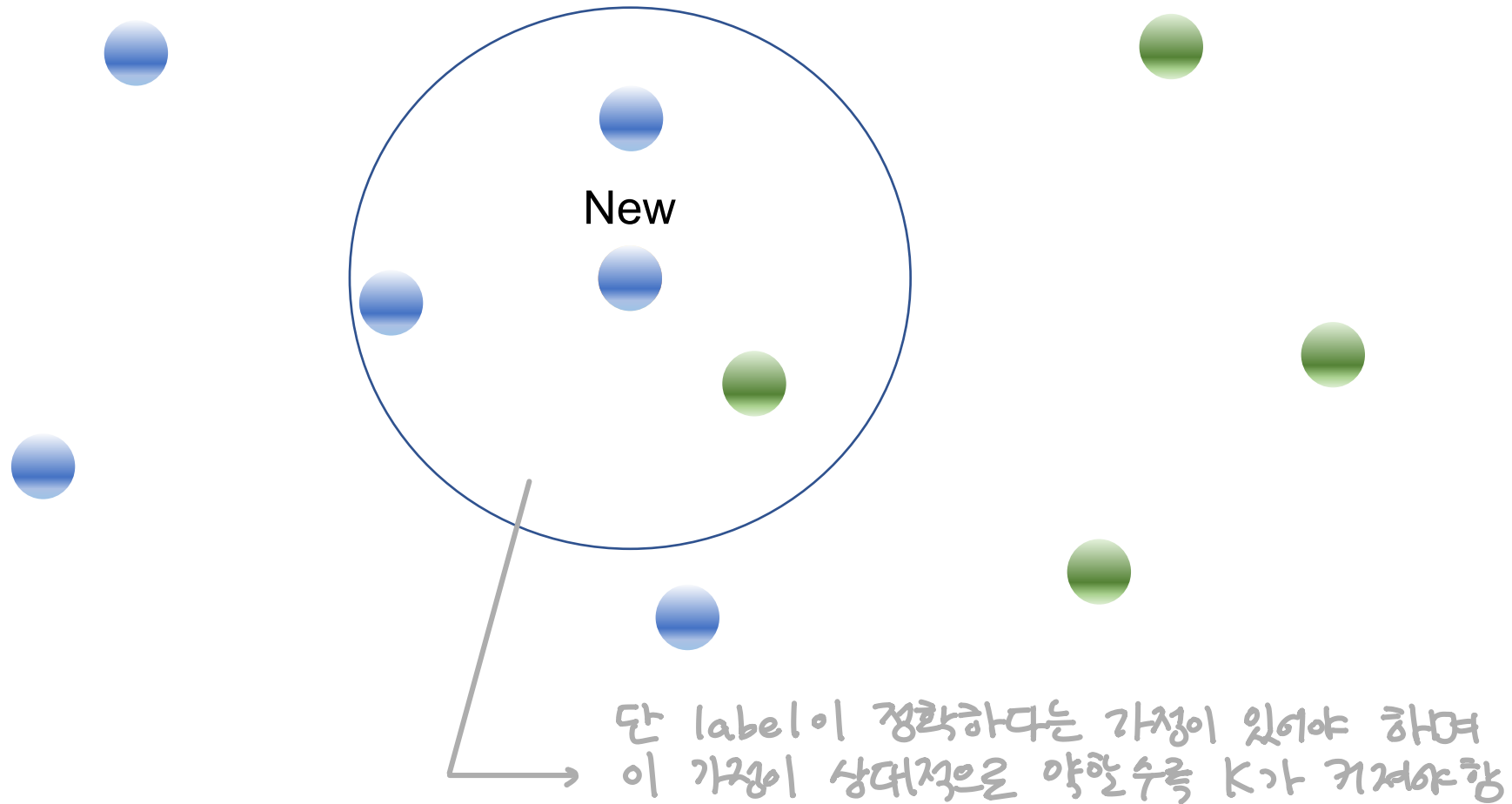
- K-Nearest Neighbor (KNN)
- Locally weighted regression

Directly generate the estimated outputs without building a model.

1-Nearest Neighbor



3-Nearest Neighbors



KNN Algorithm

1. Collect a set of data points (training data).
2. For a new data point, find k data points located nearby the new data point. 이 과정이 계산 비용이 큼
3. Return the mean value / majority class among the nearest neighbors.

Regression

Classification

Example: K-Nearest Neighbors

x_1	x_2	Class
2	2.5	1
3.5	5	1
2.5	1	1
1	3	2
6.5	4	2
4.5	6.5	2

Distance from New	
2.06	2 nd nearest
3.04	3 rd nearest
1.80	1 st nearest
3.16	4 th nearest
3.20	5 th nearest
4.53	6 th nearest

New

4	2	?
---	---	---

Characteristics of KNN Algorithm

- Instance-based Learning
 - Make a prediction based only on the observations (instances).
- Memory-based Learning
 - All training data is read into the memory space first, and then make a prediction based on it.
- Lazy Learning 학습이 없고 데이터가 입력될때 모델 없이 예측값 생성 = 공간 및 시간 복잡도 높음
 - There is no concept of 'model'.
 - The algorithm finds the near neighbors once a prediction on the new data point is requested.

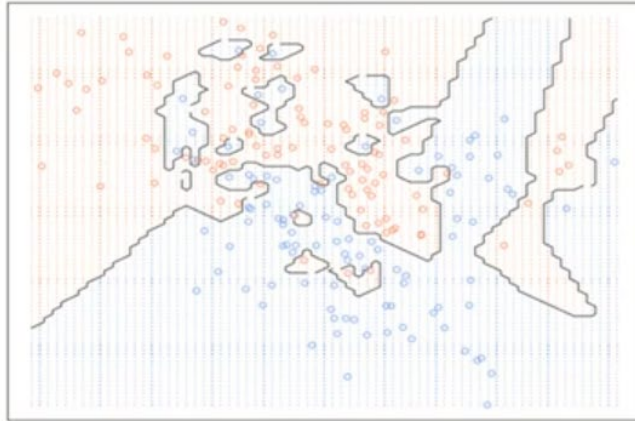
Hyper-Parameters

- What should be the value of k ?
 - Small or large?
 - Is there a universally good value?
- How should we measure the 'distance'?
 - Euclidean
 - Manhattan
 - Hamming
 - Etc.

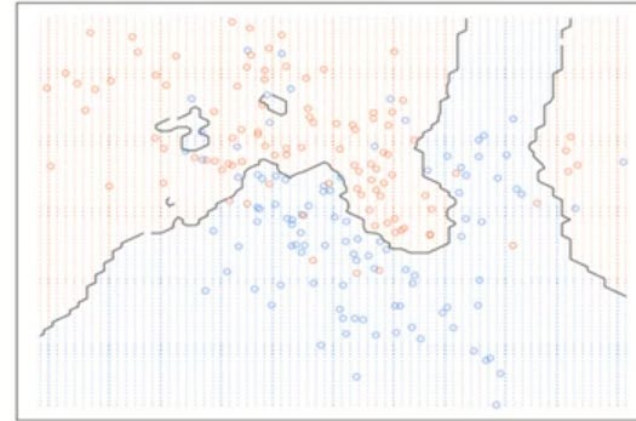


벡터 형태의 입력이 많다고 해도 유클리드 거리를 당연히 가정해서는 안 됨
각 벡터의 독립 여부/입력, 데이터 특성을 고려하여 적절히 선택해야함

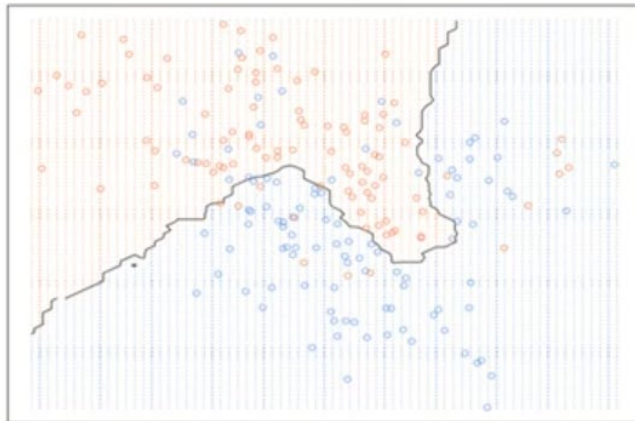
Impact of k



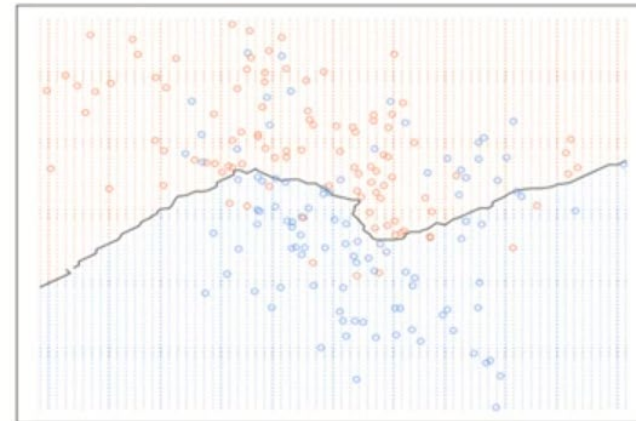
1-nearest neighbor



5-nearest neighbor



15-nearest neighbor

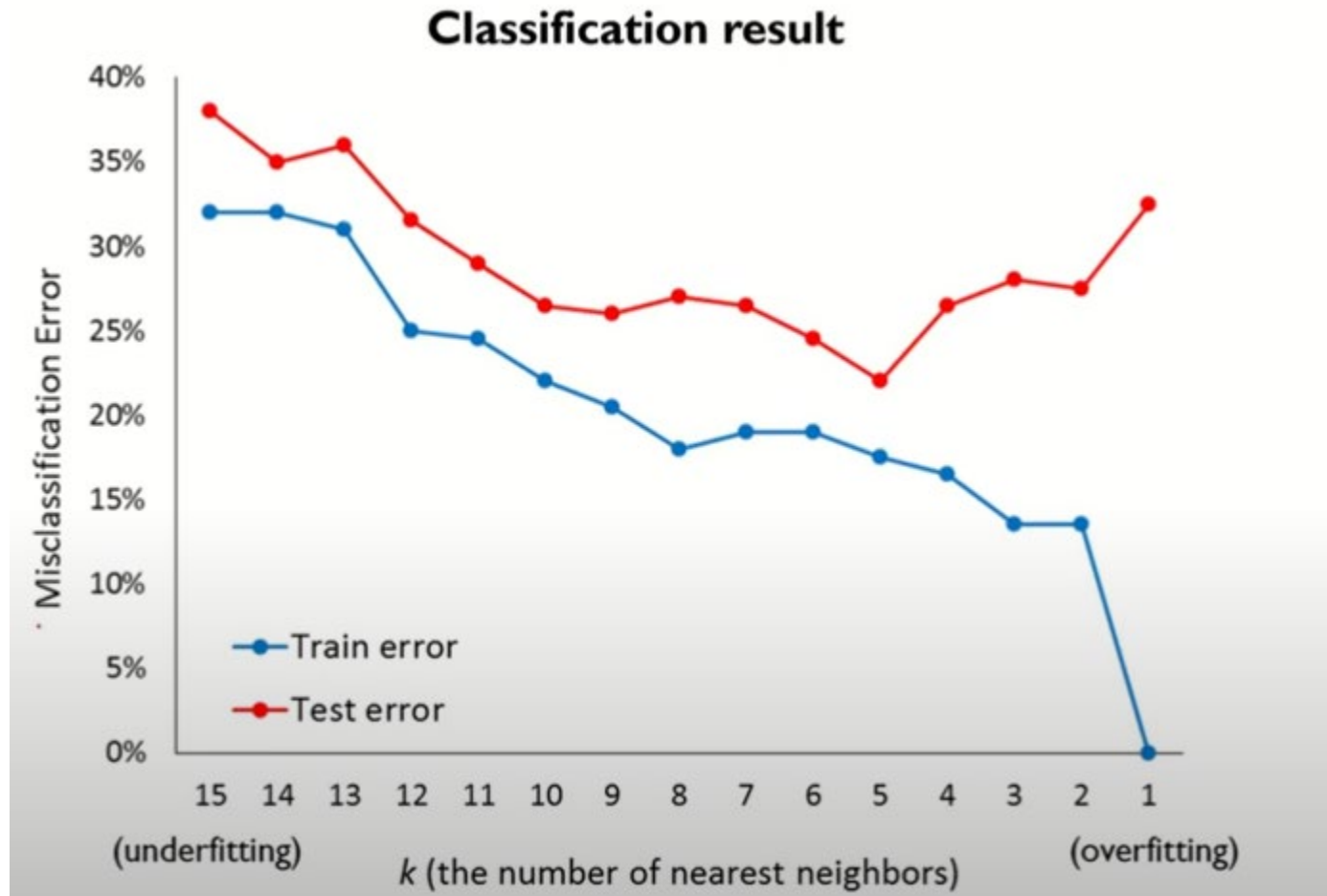


50-nearest neighbor

Brute-Force Search

- Fix a certain maximum value of k , k^* and find the best value within $(1 \sim k^*)$.
- Classification
 - Error = $\frac{1}{n} \sum_{i=1}^n I(c_i \neq \hat{c}_i)$, $I()$ is an indicator function.
- Regression
 - Error = $\frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Example: Finding k



Distance

- There is a variety of distance measures:
 - Euclidean, Manhattan, correlation, cosine similarity, etc.
 - There is no universally 'best' metric to measure the distance.
- Normalization of the data is imperative to keep few large features from dominating all the other features.
 - E.g., if data points have three features like
 - speed (10km/h ~ 100km/h)
 - weight (1.2t ~ 2.0t)
 - price (\$1,000,000 ~ \$10,000,000)

Euclidean Distance

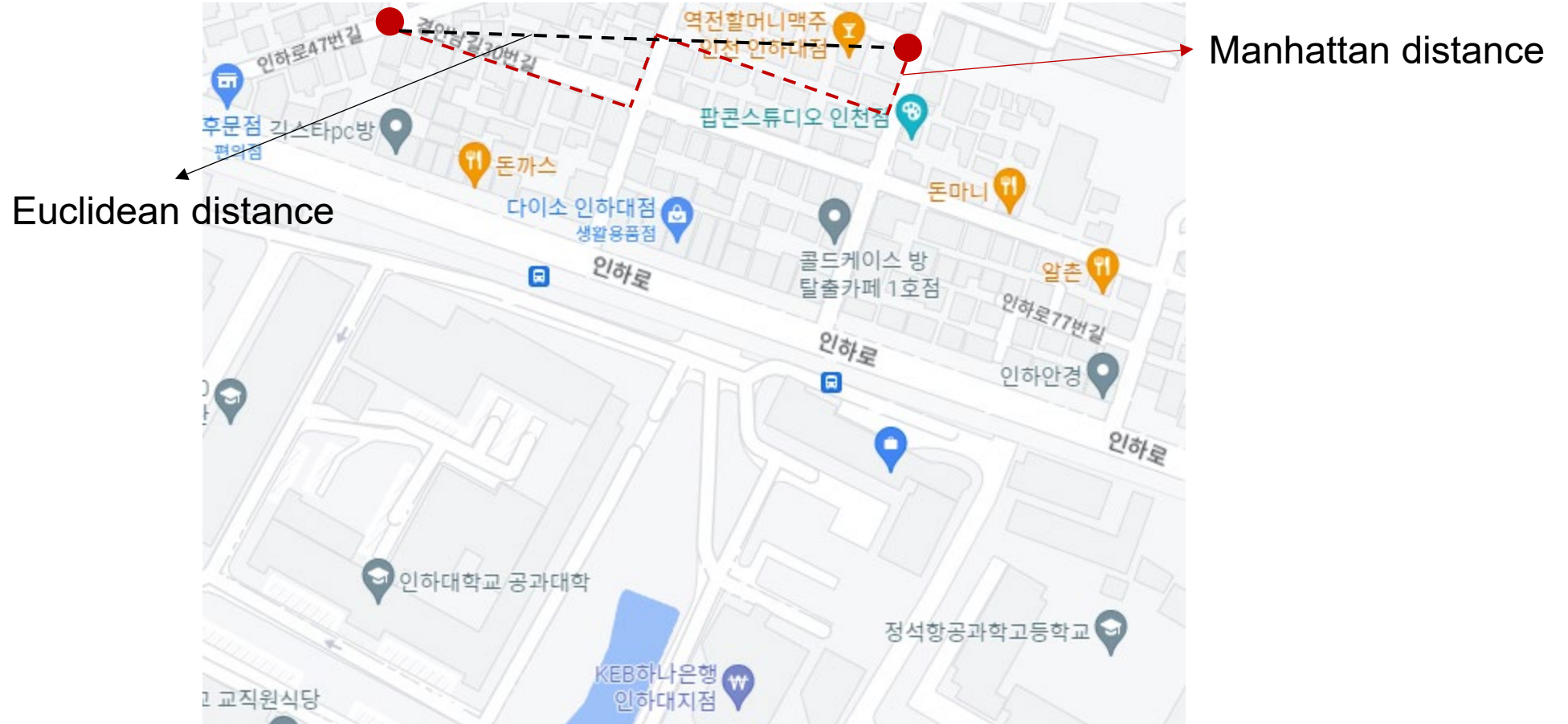
- The simplest but one of the most useful metric!
- It directly measures the geometric distance between any two data points.

$$d_{x,y} = \sqrt{\sum_{i=1}^f (x_i - y_i)^2}$$

E.g., $x = (1, 4.5, 3), y = (2, 4, 0)$, then

$$d_{x,y} = \sqrt{(1 - 2)^2 + (4.5 - 4)^2 + (3 - 0)^2} = 3.201 \dots$$

Manhattan Distance



Cosine Similarity

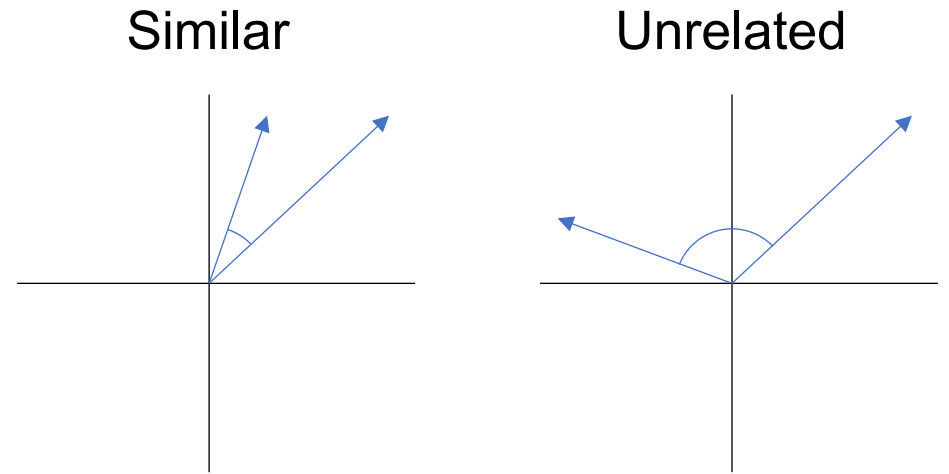
- A measure of similarity between two non-zero vectors defined in an inner product space.

$$x^T y = \|x\| \|y\| \cos \theta$$

$$d_{x,y} = \cos(\theta) = \frac{x^T y}{\|x\| \|y\|}$$

- Cosine distance

$$d_{x,y} = 1 - \cos(\theta) = 1 - \frac{x^T y}{\|x\| \|y\|}$$



KNN: Advantages & Disadvantages

- Advantages

- The performance is not strongly affected by noise. 노이즈에 강건하고 대규모 데이터셋에도 잘 동작
- Especially works well when the dataset is large.

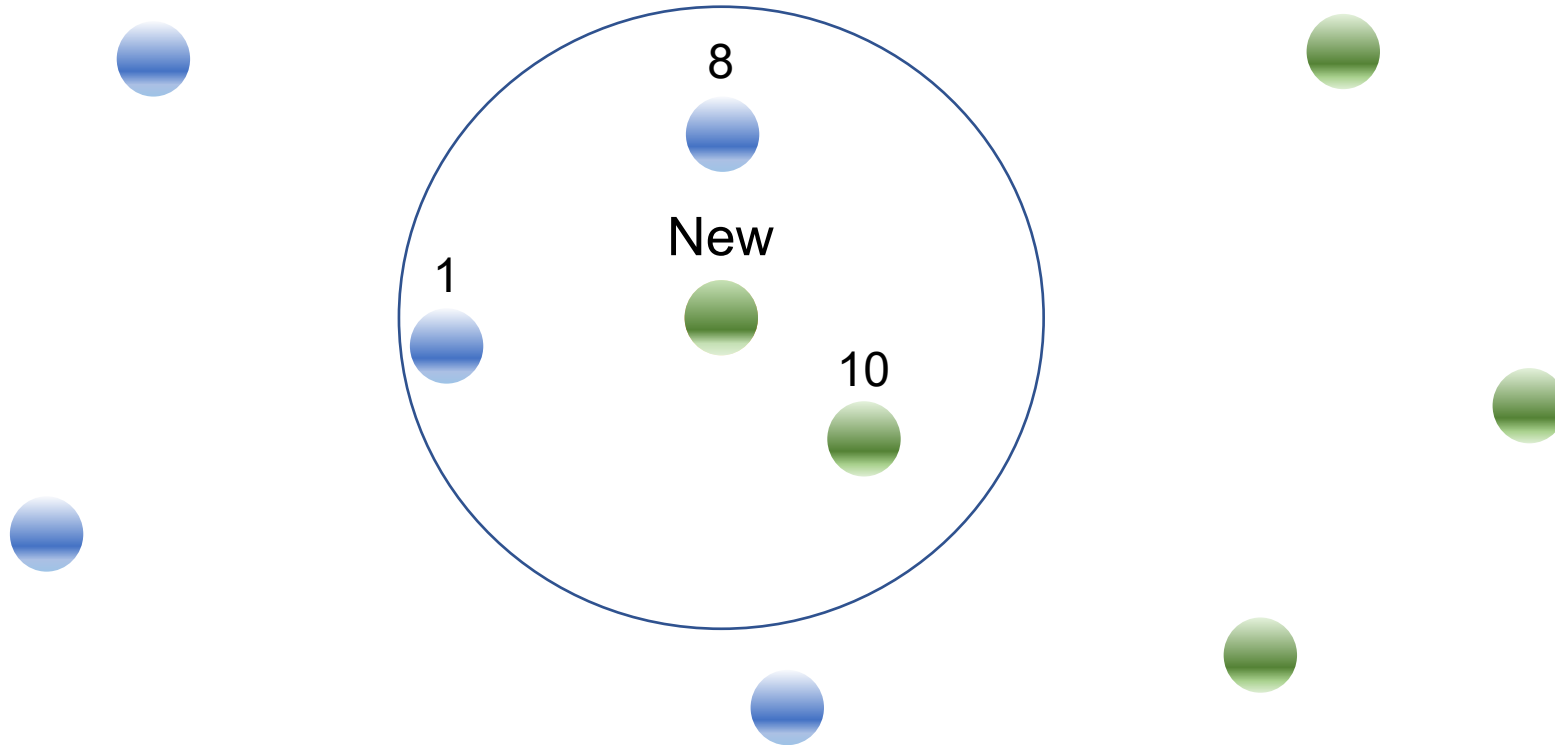
- Disadvantages

- The hyper-parameter: k K 탐색이 어려움
- The hyper-parameter: distance metric 사실상 문제에 의해 결정
- Depending on which distance metric is chosen, the calculation cost may be expensive. 계산량이 매우 클 수 있음

일반적인 KNN은 데이터의 중요도가 모두 동등하다고 가정

A Variant of KNN: Weighted KNN

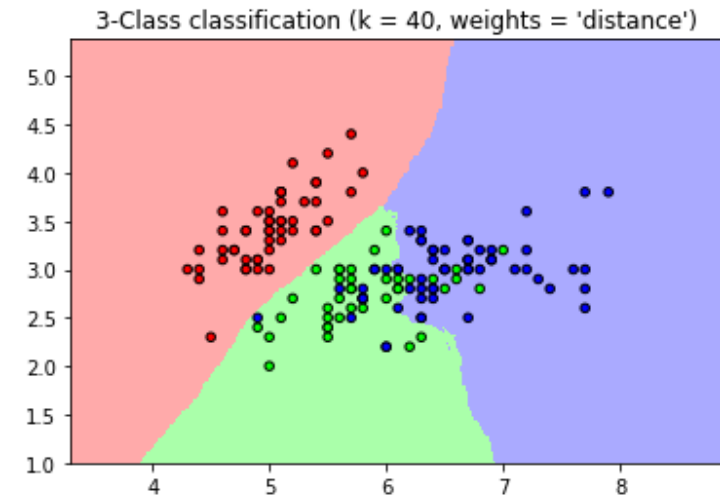
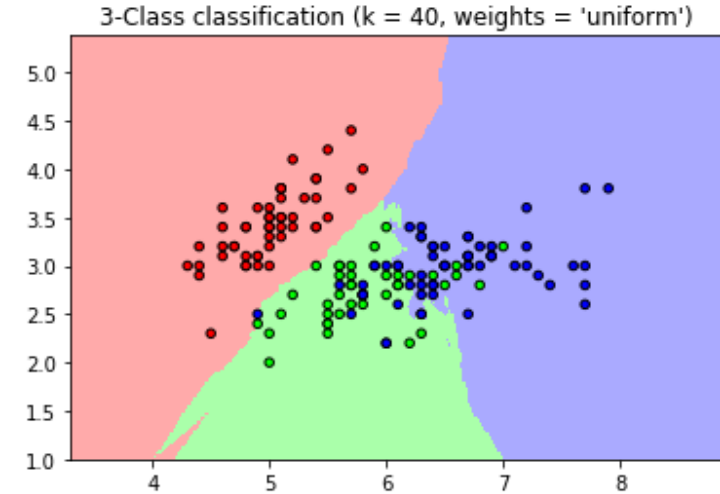
데이터의 중요도가 동등하지 않은 경우, 현실 데이터는 일반적으로 서로 동등하지 않으므로 더 최적에 가까움



E.g., the reciprocal distance can be assigned to each data as its weight factor.

Weighted KNN

- Each data point has a weight factor that is determined based on the distance metric.
- Well-defined weights make KNN more robust to the noisy data.



- K-Nearest Neighbor Algorithm
- ***Eager vs. Lazy Learning Methods***

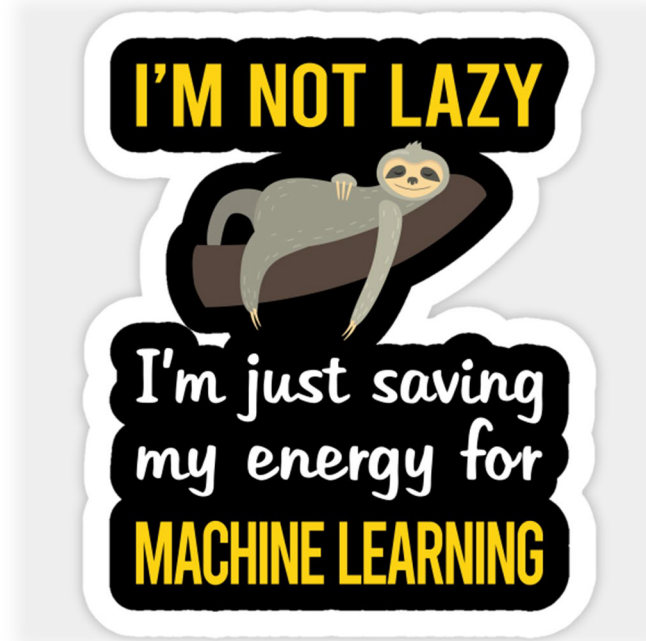
Eager Learning Methods

- A set of learning methods in which the system learns a general, input-independent target function during training.
 - E.g., SVM builds up the model based on the provided observations during training. That is, it already has determined how well it will generalize to the unknown data.



Lazy Learning Methods

- A set of learning methods that generalization of training data is deferred until an unknown query is made to the system.
 - E.g., KNN gets to know the generalization performance only after it processes the unknown query.



Comparison

- ***Eager Methods***: long training and fast prediction (inference).
 - E.g., SVM builds up the model using the whole training data points, and the prediction can be made by just calculating one quadratic equation.
 - Once the training is finished, the training data will not be used anymore.
- ***Lazy Methods***: relatively simple and the concept of training is vague...prediction always follows training.
 - E.g., KNN finds the nearest samples (training) to the unknown data.
 - It requires to store the whole training data for later inferences.

Which One is Better?

- Situation based. Sometimes, the long training time may not be allowed. Then, you can consider lazy learning methods.
- In contrast, if the application requires precise and fast inferences. Then, you should consider eager learning methods.
- For instance, what would you choose in the below situations?
 - YouTube recommender systems
 - Run-time object detectors
 - Medical data pattern recognition
 - Self-driving systems

Any Questions?

K-Nearest Neighbors (KNN) is easy to use
but powerful!

